

Lab 2: Turtle Pirate Adventure (Part



1. 📄 **Worth:** 1%
2. 📅 **Due:** ⚠️ Tuesday Oct 7 in-class to demo, Oct 7 end of day to submit online
3. 🕒 **Late submissions:** N/A
4. 📁 **Submission:** Demonstrated in-class OR submitted online

Instruction updates

⚠️ Sept 25: Given that class was cancelled on Sep 25 day, I've adjusted the lab. The main changes:

- Made this lab worth some marks
- Instructions for starting and expectations for what the code should do are much clearer.

⚠️ Sept 30: Some quick updates:

- Added instructions for drawing the red 'x'
- Added instructions for printing to console vs printing to program
- Clarified rubric

⚠️ Oct 7: Final updates

- Due date
 - Can be demo-ed in-class Oct 7
 - If demo missed, can be submitted online by Oct 7 end of day.
- Added instructions for final message (producing the grey box)

Objectives

We will learn:

- how to use python libraries (e.g. `turtle`), specifically:
 - how to use graphics for background
 - how to draw shapes and use colours
 - how to process user input
 - how to draw shapes
- how to perform mathematical operations to solve useful problems
- the difference between users and creators of software

Prerequisites

You should know the basics of the [Turtle](#) python library!

Make sure you:

- Review [the course notes about Turtle](#)
- Make sure you understand how to do things like [draw a house](#)

Background

You (a [programmer](#)) have been asked by a mighty pirate captain to create a program that trains [pirates](#) to find treasure buried in a location known only to the captain.

Your program will keep track of the coordinates of the buried treasure by storing their numbers in variables in your code.

Your program will guide a pirate to finding the treasure, keeping track of what steps they take and how far from their destination they are at each step.

Once your program is complete, it will reveal the treasure's location and tell the pirate how well they did at navigating to find the treasure.

When finished, your neighbour (a fellow programmer/[swabby](#)) will test your code to ensure it's ready for the pirates to use. Once you're ready, you'll demo the program to your teacher, who's been asked to keep track of which programs work and which don't.

The pirate captain is on a tight schedule: by the end of the week, the program must work and be easy for a pirate to use and understand. If not, the consequences could be

reduced grades on this lab... [or worse!](#)

Instructions

Write a python program that:

1. Sets the stage by creating a background based on an image, creating an icon representing the pirate's movements, and defining a hiding place for the treasure.
2. Asks the pirate running your code what position they want to start.
3. Asks them how many "steps" they want to take in a given direction, and keep tracks of the pirate's updated position as they move.
4. Informs the pirate (by writing on the screen) how far they are from the treasure every time they have moved.
5. Draws a red X on the map marking the location they travelled to.
6. Asks for more "steps" EXACTLY 8 times before pausing and showing how far from the treasure you were.
 - Ensure you inform the pirate of the updated distance, and mark the red X, at each step.
7. Finally, reveals the location of the treasure by showing an icon where the treasure was located.

Instructions for each step follow in the sections below.

0. Getting started

Like all labs and assignments, you'll need to:

- Create a Pycharm project in your SN1 OneDrive folder for lab-2
- Create a python file inside that project (name it `treasure-hunt.py`)
- Download `lab-2-images.zip` and extract it. Make sure the image files are in the same folder as your `treasure-hunt.py` file.

Make sure your `treasure-hunt.py` program begins with the following code:

```
import turtle
```

Make sure your `treasure-hunt.py` program end with the following code:

```
# IMPORTANT: don't have any code below this line, it won't run.
turtle.exitonclick()
```

1. Set the stage

Once we've imported the turtle library, we can now set the stage for our treasure hunt simulation:

- Determine the location of the treasure and store it in two variables (X_TREASURE, Y_TREASURE)
- Set the background image
- Create a turtle representing the pirate with a custom icon

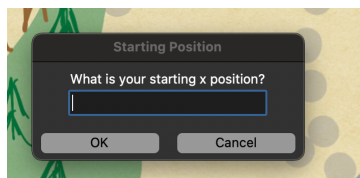
You can find examples for using the turtle library to complete each of these tasks in the [course notes](#).

You can find the background image and the icon I used for the pirate in the `lab-2-images.zip`, namely `island-bg.png` and `pirate-24x24.gif` respectively.

2: Ask the pirate to choose a starting position

Use `turtle.numinput` to ask the pirates for:

- the starting x-coordinate position.
- the starting y-coordinate position.



Store each these values in a variable. You can find [course notes on using](#)



The program with the background set, and the pirate turtle (with a custom icon) beginning at [0, 0]

Turtle number input.

in the middle of the map.

3. Inform the pirate of their current distance to treasure

Move the turtle to its new starting position based on the input from the user. Then, calculate the distance between the pirate and the treasure. Store that distance in a variable.

You will find the Pythagorean Equation useful for this calculation:

$$distance = \sqrt{(x_{pirate} - x_{treasure})^2 + (y_{pirate} - y_{treasure})^2}$$

Then, **display the distance from the treasure next to the pirate.**

You can use the same turtle as the pirate to print the pirate's location. You can also create a new turtle using `turtle.clone()`.

3.5 Inform the programmer of the progress in the program so far

After each pirate step, you should also use your variables and the `print()` function to update the programmer (NOT the pirate) about the current state of the program. For example:

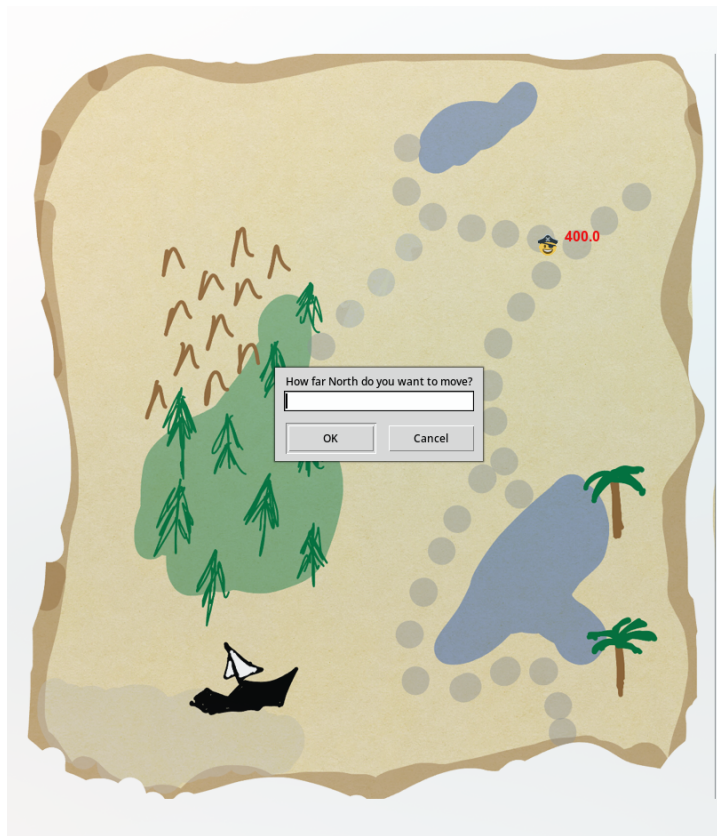
Step 1:

Treasure position: (-200, 200)

Pirate position: (200, 200)

Distance between pirate

Distance travelled so far



The program prompting for the first North move, with the start location set to [200, 200]. In this case, the treasure is located at [-200, 200], so the distance is 400.0

4. Ask the pirate to move

Using the `turtle.numinput` function, prompt the user for the distance that they want to move.

Move the turtle

- Note that by default, the turtle is facing west.
- Make sure the turtle is facing north when you are finished.

After moving the turtle

- have the turtle turn left by 90 degrees
- Update the current position of the pirate (`x_pirate` , `y_pirate`) after each movement

Update the console as well with the new positions:

```
Step 2:
Treasure position: (-200, 200)
Pirate position: (200.0, 300.0)
Distance between pirate and treasure: 412.31
Distance travelled so far: 100.00
```

5. Draw a red X marking visited locations

(Advanced: leave this task for last).

Mark each location that the pirate has visited by drawing a red x!

There are a few ways to do this:

- Have your pirate turtle (or a [clone of the pirate turtle](#)) draw a red X by moving at 45 degree angles
- Add red "X" `.gif` shape to your turtle shape collection, then use a clone to make a `stamp()` with that new shape at the pirate's position.

6. Ask for 8 steps in total

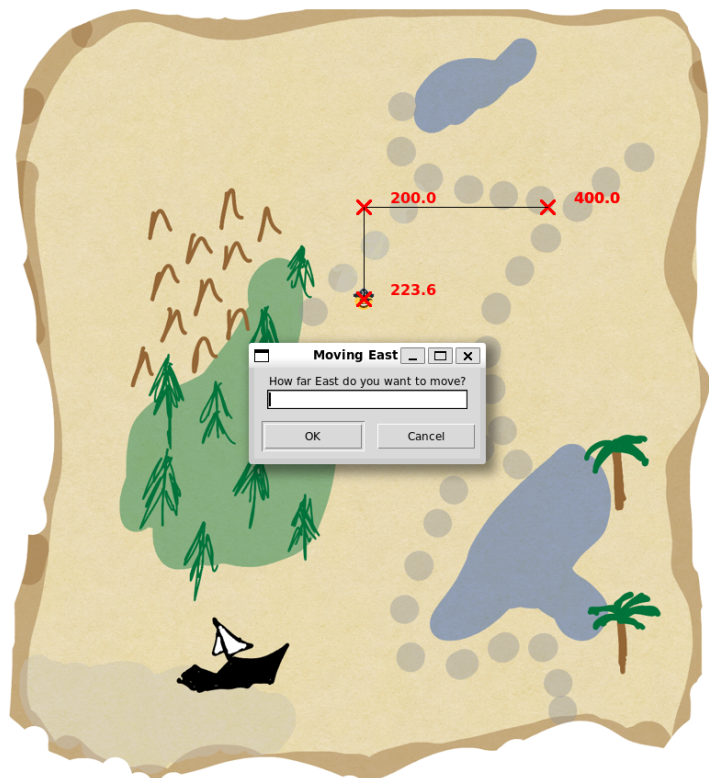
As is well known, pirates can only turn left. The pirates must always follow a *counter-*

clockwise order of moves:

- North
- then West
- then South
- then East
- North again
- then West
- then South
- finally East

This means you have to repeat the process of asking the pirate for a step, calculating their distance, and printing the result, **8 times**. It's a bit tedious!

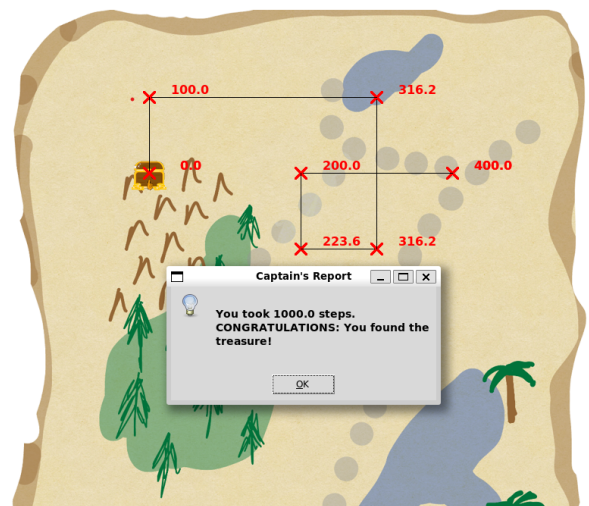
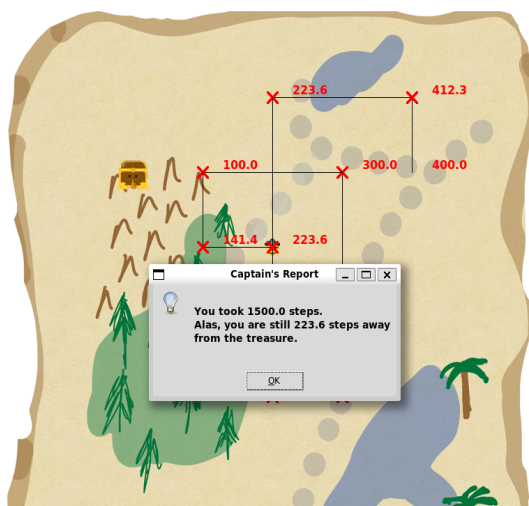
I recommend making sure you can get the first four steps correct, then using [copy-paste](#) for the final four.



After a move of 0 in the North direction, a move of 200 in the West direction, and 100 in the south direction, we can see the map has been updated with X characters and distances for each location.

7. Reveal treasure and post results

In the end, after 8 steps, display how many steps the pirates traveled in total. How far are they from the treasure at the end?





Two different game plays with the treasure located at $[-200, 200]$. You should tell the pirate whether they succeeded or not!

Note: the “reveal” of the treasure is using a turtle and the `treasure.gif` image included in your lab starter files.

⚠ To display the message box like this, you can use code like follows:

```
# You can make a message variable to keep things neat
message=""
You took 1 million steps
You found the treasure!
""

# use turtle.TK.messagebox.showinfo to produce a message box
turtle.TK.messagebox.showinfo(title="Captain's Report", message=message)
```

Demoing your work

Once you’re confident your program is working well, you should *test* your program by letting someone else use it!

It’s always best to try it out on real pirates. Swap seats with the person next to you and try running each others’ program! You are now the pirate and playing someone else’s game. Your goal should be to find a bug in their program 🐞

You can assume that the user will only type integer numbers into your program.

Rubric

When you’ve tried the program out with your partner and it seems to be working pretty well, call the teacher for an in-person demo to be graded in-class.

Checklist:

- ☐ Have you chosen the correct background image and updated the turtle icon?
- ☐ Does your program ask the pirate where to start, and place them correctly?
- ☐ Does your program ask the pirate for the number of steps, and move them in the

correct direction?

- ☐ Does your program calculate and display the distance at each step correctly?
- ☐ Does your program ask for 8 steps in total, starting with North and proceeding counter-clockwise?
- ☐ Does your program print the locations of the pirate and the distance travelled so far in the console correctly?
- ☐ (Advanced) Does your program print a red X at each step?

References

Pirate Emoji:

https://moreemoji.allezsoyez.com/	
Author	alleZSoyez, derivative mork of Twemoji

Treasure Emoji:

[png image from pngtree.com/](#)